

Notebook 9

In this notebook we will try to find the heating model from a cylindrical plate.

Here we will see how we can use a Neural Network to find out how the cylindrical plate is heated. We will use the same strategy as in the previous notebooks. We will first solve the governing equation get the data, T_i . Then given the data, we will replace the heating function with a Neural network to see if we can tune the coefficients of the Neural network to fit the data. WARNING: The code in this notebook is computationally expensive. This notebook will take much longer to run than the other notebooks in this course.

The governing equation for the temperature distribution is given by

$$\frac{d^2T}{dr^2} + \frac{1}{r} \frac{dT}{dr} = d(T^4 - T_R^4) - g(r)$$

You are given that $r \in [0, 5]$, $d = 0.1$, $T_R = 10$ and $g(r) = (r - 5)^2$.

Use the finite difference approximation

$$\frac{d^2T}{dr^2} \approx \frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta^2}$$

$$\frac{dT}{dr} \approx \frac{T_{i+1} - T_{i-1}}{2\Delta}$$

We substitute back into the original equation to get

$$\frac{T_{i-1} - 2T_i + T_{i+1}}{\Delta^2} + \frac{1}{r_i} \frac{T_{i+1} - T_{i-1}}{2\Delta} = d(T_i^4 - T_R^4) - g(r_i)$$

This equation is nonlinear (due to the T^4 term, we cannot put it into matrix form). For a grid with 6 data points, we have

$$(T_2 - T_1)/\Delta = 0 \rightarrow T_2 - T_1 = 0$$

$$\frac{T_1 - 2T_2 + T_3}{\Delta^2} + \frac{1}{r_2} \frac{T_3 - T_1}{2\Delta} = d(T_2^4 - T_R^4) - g(r_2)$$

$$\frac{T_2 - 2T_3 + T_4}{\Delta^2} + \frac{1}{r_3} \frac{T_4 - T_2}{2\Delta} = d(T_3^4 - T_R^4) - g(r_3)$$

$$\frac{T_3 - 2T_4 + T_5}{\Delta^2} + \frac{1}{r_4} \frac{T_5 - T_3}{2\Delta} = d(T_4^4 - T_R^4) - g(r_4)$$

$$\frac{T_4 - 2T_5 + T_6}{\Delta^2} + \frac{1}{r_5} \frac{T_6 - T_4}{2\Delta} = d(T_5^4 - T_R^4) - g(r_5)$$

$$T_6 = 10 \rightarrow T_6 - 10 = 0$$

We will take the same strategy as the previous notebooks. We will first solve the forward model to get our data and then use the data to see if it can help us tune a Neural Network to give us back the heating model.

```
In [1]: using NonlinearSolve
using CairoMakie
using Random
using Distributions
using Lux
using ComponentArrays

using Optimization
using Optimisers
using OptimizationOptimisers
using ForwardDiff
```

```
In [2]: d=0.2;TR=10;R=5;
Delta=0.1;
r=0:Delta:R;
```

Set up parameters of the problem

```
In [1]: N=length(r); #n is the number gridpoints
T=10*ones(N);
p=(r[2:N-1].-R).^2 # this is the ground truth of the heating model
```

UndefVarError: `r` not defined

Stacktrace:

[1] top-level scope

@ ~/Library/CloudStorage/OneDrive-TheUniversityofMelbourne/JuliaPrograms/CSIRO_Workshop/jl_notebook_cell_df34fa98e69747e1a8f8a730347b8e2f_W5sZmlsZQ==.jl:1

Define the nonlinear vector functions. In the function below `p()` is the heating model.

```
In [4]: function HeatedDiskExample(T,p)
    func=vcat( T[1]-T[2],
               ((T[3:N]-2*T[2:N-1]+T[1:N-2])/Delta^2.0).+(1/(2*Delta*r[2:N-1]))*
               T[N]-TR)
    return func
end
```

HeatedDiskExample (generic function with 1 method)

```
In [5]: HeatedDiskExample(T,p)
```

```
51-element Vector{Float64}:
 0.0
24.010000000000005
23.04
22.090000000000003
21.159999999999997
20.25
19.360000000000003
18.49
17.64
16.81
⋮
 0.6400000000000001
 0.48999999999999994
 0.36
 0.25
 0.16000000000000003
 0.09
 0.04000000000000001
 0.010000000000000002
 0.0
```

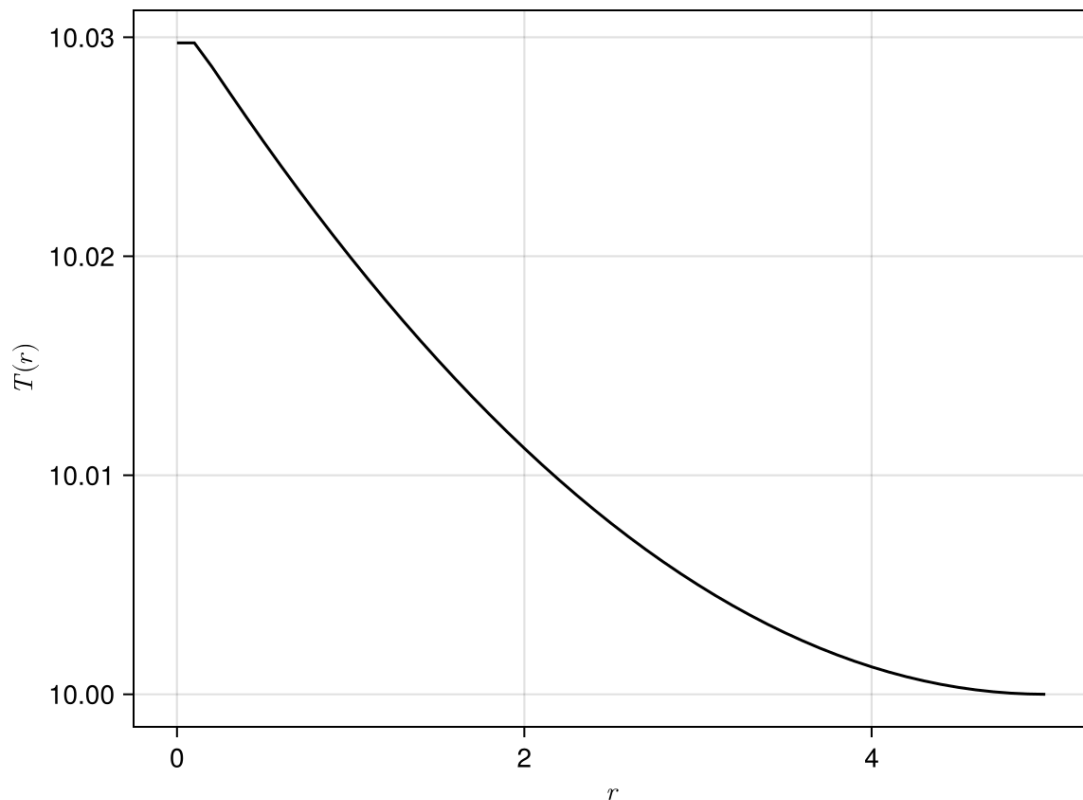
Use the `NonlinearSolve()` package in Julia code to solve the problem.

```
In [6]: T0=10*ones(length(r)) #initial guess
        prob = NonlinearProblem(HeatedDiskExample,T0,p)
        sol = NonlinearSolve.solve(prob)
```

```
retcode: Stalled
u: 51-element Vector{Float64}:
10.029743069170305
10.029743069170305
10.028664132342852
10.027498921770123
10.02634700196618
10.025218382474442
10.02411408988066
10.023034249043842
10.021978893861958
10.020948048635198
⋮
10.00080142350397
10.000613963832759
10.000451490255656
10.000314006414923
10.000201515287696
10.000114018264396
10.000051506028191
10.000013868272578
10.0
```

Plot the solution

```
In [7]: fig = Figure() #set up a figure
        ax1 = CairoMakie.Axis(fig[1, 1], xlabel = L"r", ylabel = L"T(r)") #Define
        lines!(ax1,r,sol.u;color=:black) #Plot i_5 vs V_1
        display(fig)
```



```
CairoMakie.Screen{IMAGE}
```

```
In [8]: Tdata=sol.u
```

```
51-element Vector{Float64}:
 10.029743069170305
 10.029743069170305
 10.028664132342852
 10.027498921770123
 10.02634700196618
 10.025218382474442
 10.02411408988066
 10.023034249043842
 10.021978893861958
 10.020948048635198
  ⋮
 10.00080142350397
 10.000613963832759
 10.000451490255656
 10.000314006414923
 10.000201515287696
 10.000114018264396
 10.000051506028191
 10.000013868272578
 10.0
```

So this is our training data

Now we will try and see if we can get the heating term $g(r)$ from the data and using a Neural Network to model $g(r)$. Set up the architecture of the Neural Network

In [9]:

```
rng=Xoshiro(1)

gModel = Chain(Dense(1 => 3,tanh),
Dense(3 => 3,tanh),
Dense(3=>1))

# Initialize the model weights and state
parameters, layer_states = Lux.setup(rng, gModel)

((layer_1 = (weight = Float32[-2.4690006; -2.4631705; -2.5724392;;], bias
= Float32[-0.94601214, 0.39765334, -0.9980464]), layer_2 = (weight = Float
32[1.3124168 -1.023973 -0.3902467; 1.3830966 -1.0305337 0.93506414; 0.5980
048 0.90060115 0.9236896], bias = Float32[-0.07401537, -0.38369304, -0.483
11597]), layer_3 = (weight = Float32[0.45988524 -0.09438288 0.27005172], b
ias = Float32[0.031401504])), (layer_1 = NamedTuple(), layer_2 = NamedTupl
e(), layer_3 = NamedTuple()))
```

Set up `loss` for the optimiser

In [10]:

```
function loss(param,Tdata)
    T0=10*ones(length(r)) #initial guess
    gvals,_=gModel(r',param,layer_states)
    p=gvals[2:end-1]
    prob = NonlinearProblem(HeatedDiskExample,T0,p)
    TNN = NonlinearSolve.solve(prob)
    return sum((TNN.-Tdata).^2)
end
```

`loss` (generic function with 1 method)

In [11]:

```
function callback(state, l)
    push!(losses, l)
    if length(losses) % 50 == 0
        println("Current loss after $(length(losses)) iterations: $(losse
    end
    return false
end

losses = Float64[]
adtype = AutoForwardDiff()
optf = OptimizationFunction(loss, adtype)

#optprob = OptimizationProblem(optf,ComponentArray(sol.u),Float32.(ydata)
optprob = OptimizationProblem(optf,ComponentArray(parameters),Float32.(Td
sol=Optimization.solve(optprob,Adam(0.1f0),maxiters=2000, callback=callba
```

```

Current loss after 50 iterations: 0.004402133170515299
Current loss after 100 iterations: 0.004381922539323568
Current loss after 150 iterations: 0.004381722304970026
Current loss after 200 iterations: 0.004381717648357153
Current loss after 250 iterations: 0.004381714854389429
Current loss after 300 iterations: 0.00438171299174428
Current loss after 350 iterations: 0.004381711129099131
Current loss after 400 iterations: 0.004381709732115269
Current loss after 450 iterations: 0.004381708335131407
Current loss after 500 iterations: 0.004381707403808832
Current loss after 550 iterations: 0.00438170600682497
Current loss after 600 iterations: 0.004381704609841108
Current loss after 650 iterations: 0.004381702747195959
Current loss after 700 iterations: 0.00438170088455081
Current loss after 750 iterations: 0.004381698556244373
Current loss after 800 iterations: 0.004381694830954075
Current loss after 850 iterations: 0.0043816897086799145
Current loss after 900 iterations: 0.004381680395454168
Current loss after 950 iterations: 0.004381660372018814
Current loss after 1000 iterations: 0.0043816049583256245
Current loss after 1050 iterations: 0.004381360020488501
Current loss after 1100 iterations: 0.001199397724121809
Current loss after 1150 iterations: 0.0007504819659516215
Current loss after 1200 iterations: 0.0007113250903785229
Current loss after 1250 iterations: 0.0006452464149333537
Current loss after 1300 iterations: 0.0005979260895401239
Current loss after 1350 iterations: 0.0005521479761227965
Current loss after 1400 iterations: 0.00046322308480739594
Current loss after 1450 iterations: 0.00012341640831436962
Current loss after 1500 iterations: 7.30333777028136e-5
Current loss after 1550 iterations: 5.2420044085010886e-5
Current loss after 1600 iterations: 2.032873271673452e-5
Current loss after 1650 iterations: 1.4288574675447308e-5
Current loss after 1700 iterations: 1.1098155482613947e-5
Current loss after 1750 iterations: 8.595382496423554e-6
Current loss after 1800 iterations: 9.857067198026925e-6
Current loss after 1850 iterations: 5.9170506574446335e-6
Current loss after 1900 iterations: 8.114351658150554e-6
Current loss after 1950 iterations: 4.458422608877299e-6
Current loss after 2000 iterations: 5.423477432486834e-6

```

retcode: Default

```

u: ComponentVector{Float32}(layer_1 = (weight = Float32[-0.494464; -0.9707
485; -3.4914603;], bias = Float32[0.629945, 1.5787249, -2.9110053]), layer_2 = (weight = Float32[2.6358047 -0.8795146 -0.3748974; -0.33105648 0.071
77882 5.1845593; -1.5579252 1.0068098 3.6773732], bias = Float32[-0.005386
1965, -5.8795314, -4.680301]), layer_3 = (weight = Float32[14.529892 -4.32
119 -3.9911258], bias = Float32[4.234879]))

```

Compute the solution with latest parameters of the Neural Network to see if we have manage to compute the solution.

```

In [12]: T0=ones(length(r)) #initial guess
gvals,_=gModel(r',sol.u,layer_states)
p=gvals[2:end-1]
prob = NonlinearProblem(HeatedDiskExample,T0,p)
TNN = NonlinearSolve.solve(prob)

```

```

└ Warning: Mixed-Precision `matmul_cpu_fallback!` detected and Octavian.jl
cannot be used for this set of inputs (C [Matrix{Float64}]: A [Base.Reshap
edArray{Float32, 2, SubArray{Float32, 1, Vector{Float32}, Tuple{UnitRange
{Int64}}, true}, Tuple{}}] x B [LinearAlgebra.Adjoint{Float64, StepRangeLe
n{Float64, Base.TwicePrecision{Float64}, Base.TwicePrecision{Float64}, Int
64}}]). Falling back to generic implementation. This may be slow.
└ @ LuxLib.Impl /Users/asho/.julia/packages/LuxLib/zPBrt/src/impl/matmul.j
l:194

```

```
retcode: Stalled
```

```
u: 51-element Vector{Float64}:
```

```

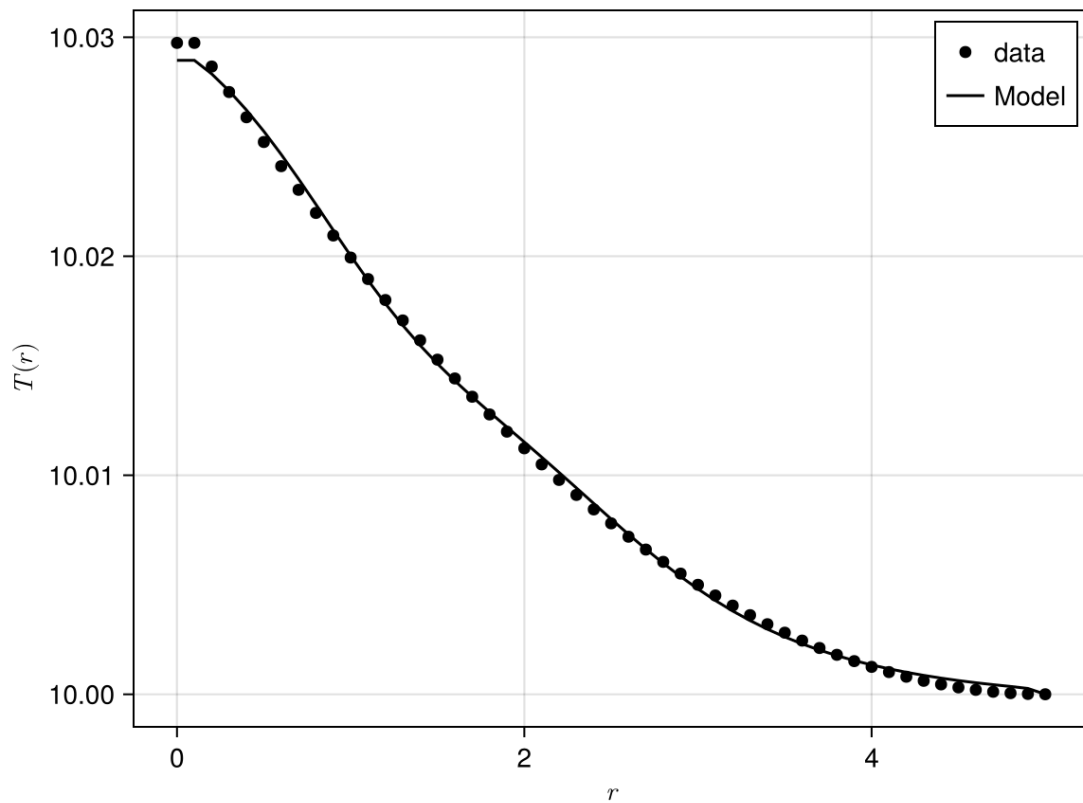
10.02894680126863
10.02894680126863
10.028325192642294
10.027552583989088
10.026676061642696
10.025704937005235
10.024650500192953
10.02353029020943
10.022367748779564
10.021190290085503
⋮
10.001004544425497
10.000863814498612
10.000738522624449
10.000626897382151
10.00052735303614
10.000438334845184
10.000356952832155
10.000265624116372
10.0

```

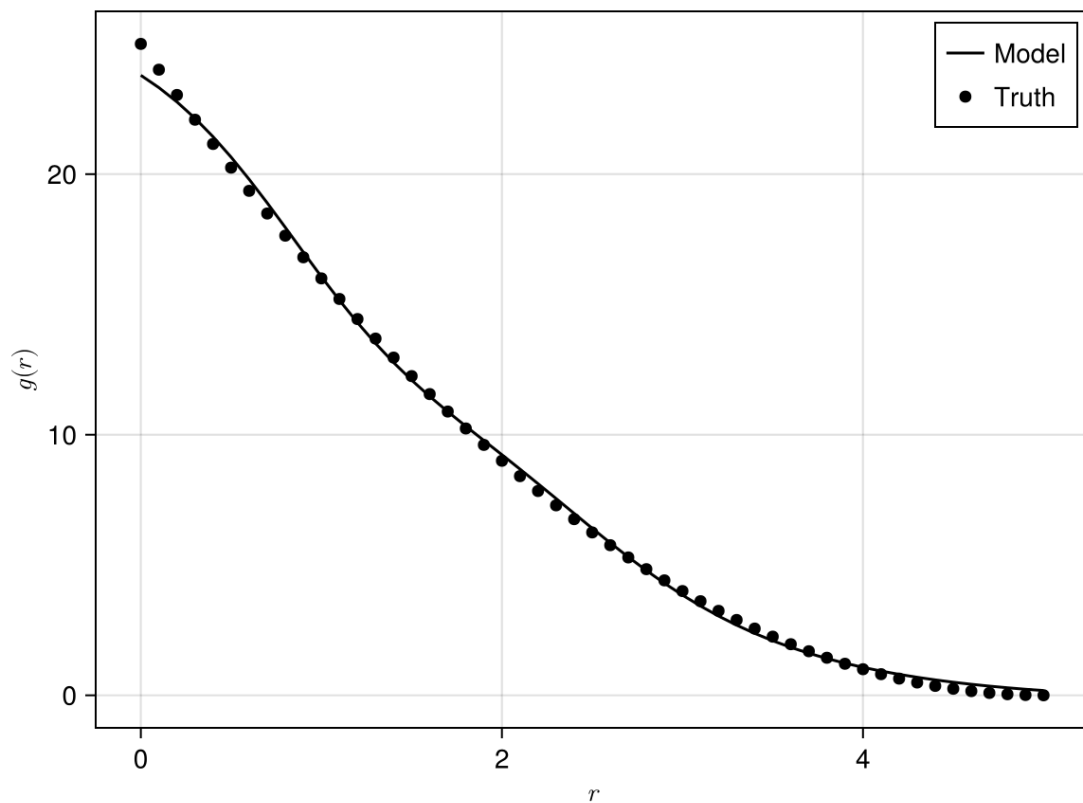
```

In [13]: fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"r", ylabel = L"T(r)")
scatter!(ax, r, Tdata, color=:black, label="data")
lines!(ax, r, TNN, color=:black, label="Model")
axislegend(ax, position=:rt)
fig

```



```
In [14]: fig = Figure()
ax = CairoMakie.Axis(fig[1, 1], xlabel = L"r", ylabel = L"g(r)")
lines!(ax, r, gvals[:], color=:black, label="Model")
scatter!(ax, r, (r.-R).^2, color=:black, label="Truth")
axislegend(ax, position=:rt)
fig
```



Now that the Neural Network is a good approximation of $g(r)$, we can use it to predict $T(r)$ when other parameters change.